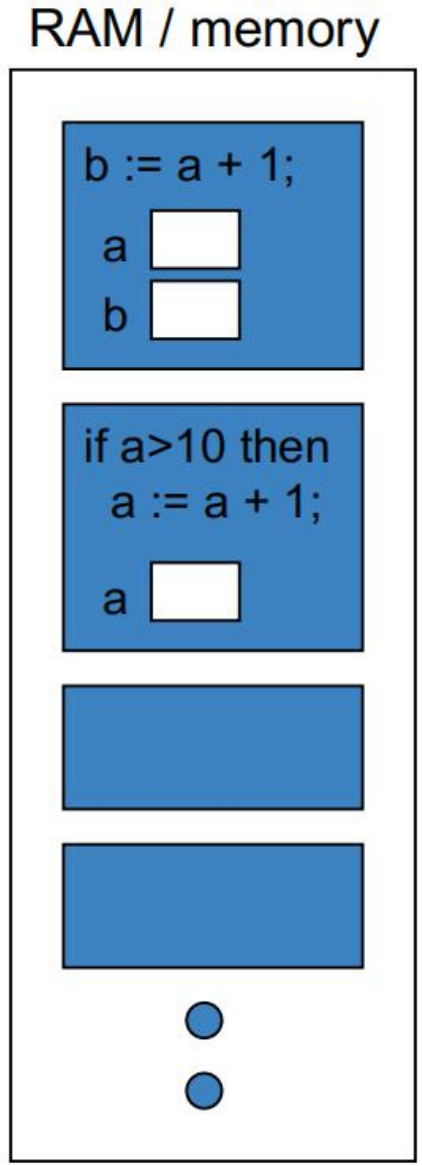
COMP2113

Memory Management

1. **Partition, paging and segmentation**
2. *Virtual memory*
3. *Address translation and page fault handling*

Memory Management

- Subdivide memory to accommodate multiple processes
- Allocate memory efficiently – pack as many processes into memory as possible

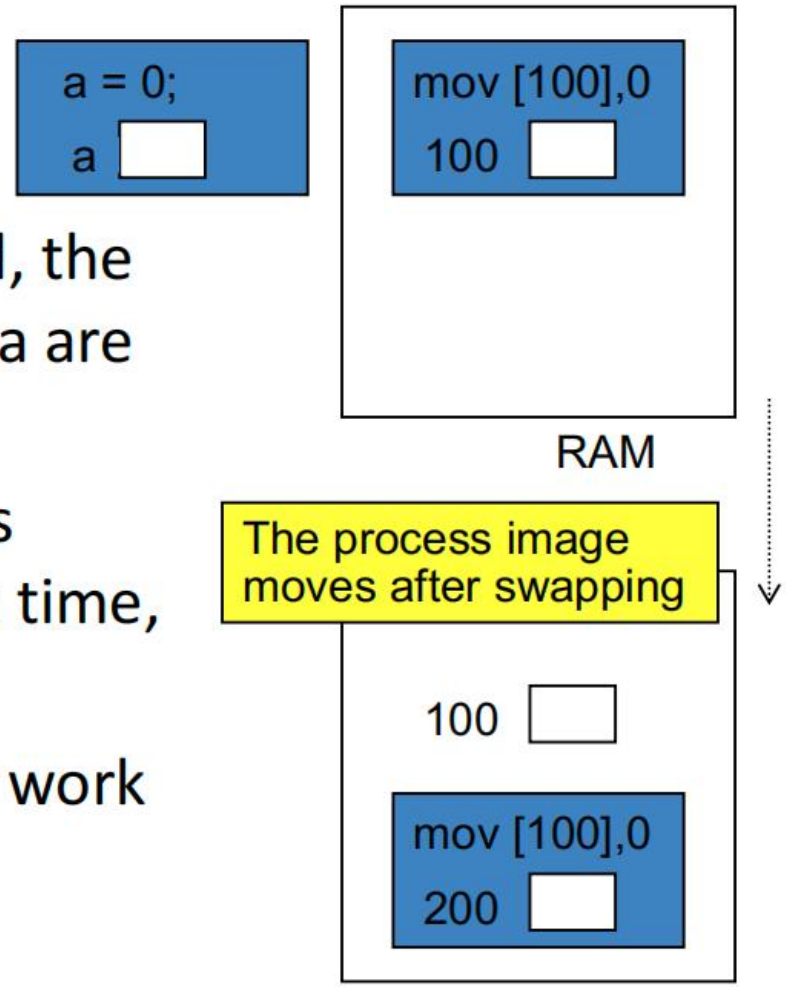


Memory Management Requirements

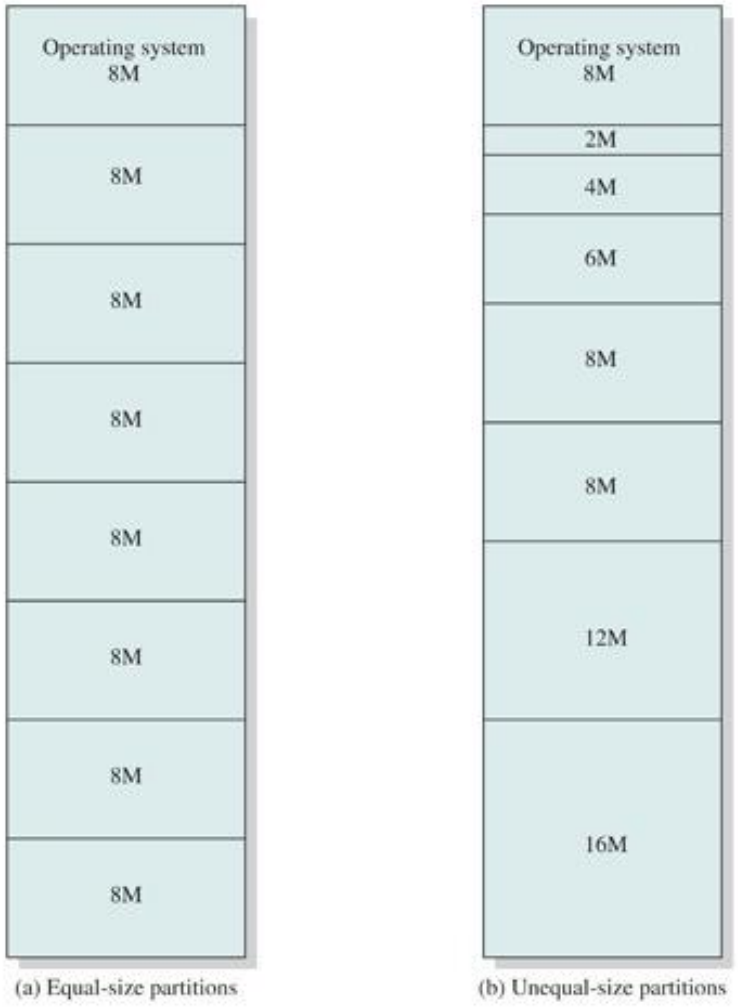
- Relocation
- Protection
- Sharing
- Logical organization
- Physical organization

Memory Management Requirements

- Relocation
  - when a program is compiled, the address of the code and data are fixed
  - the memory range a process occupied may change in run time, e.g. due to swapping.
  - The program may no longer work afterward



Partition- Fixed Partitioning



Definition

| Main memory (excluding OS area) pre-split into partitions at system boot time.

| Partition boundaries are static, unchanged during runtime.

Example of Fixed Partitioning of a 64-Mbyte Memory

Partition- Fixed Partitioning

Fixed

Equal-size

Rule: any process ≤ partition size can occupy one partition
When all partitions full → perform swapping

Problem 1: Program larger than partition → requires Overlays
Problem 2: Internal Fragmentation (unused wasted space inside a partition, small program takes whole partition)

Unequal-size

(still FIXED, preset sizes, NOT dynamic partition)

Improvement: alleviate above two problems 1&2 BUT NOT fully solved
- Larger partitions accommodate big programs
- Smaller partitions reduce degree of internal fragmentation

Two placement queue strategies

Strategy A: One dedicated queue per partition
- Policy: assign process to the smallest partition that can fit it
- Advantage: minimise internal fragmentation for individual partition
- Drawback: sub-optimal for whole-system memory utilization

(large partition may stay idle even if small jobs exist)

Strategy B: Single global queue (preferred approach)
- Policy: select smallest available partition that can hold incoming process
- When all partitions occupied: make swapping-out decision
- Swap-out considerations: process state(blocked / ready), priority

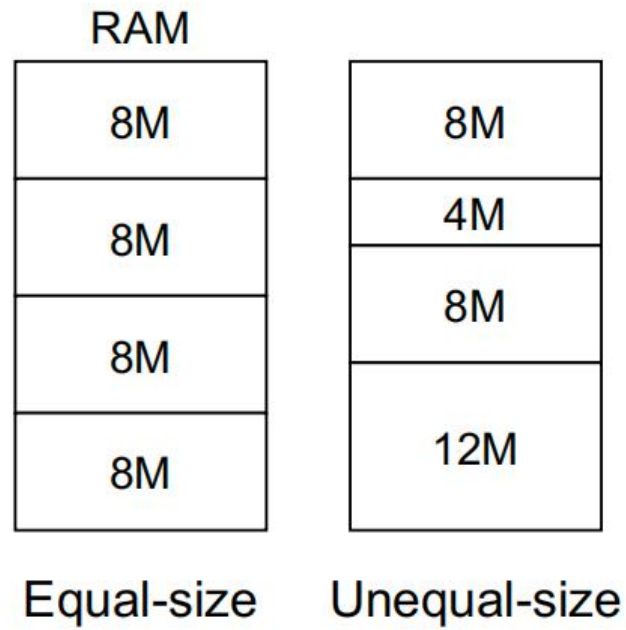
Overall Pros & Cons of Fixed Partitioning

Pros: Simple implementation, low OS runtime overhead

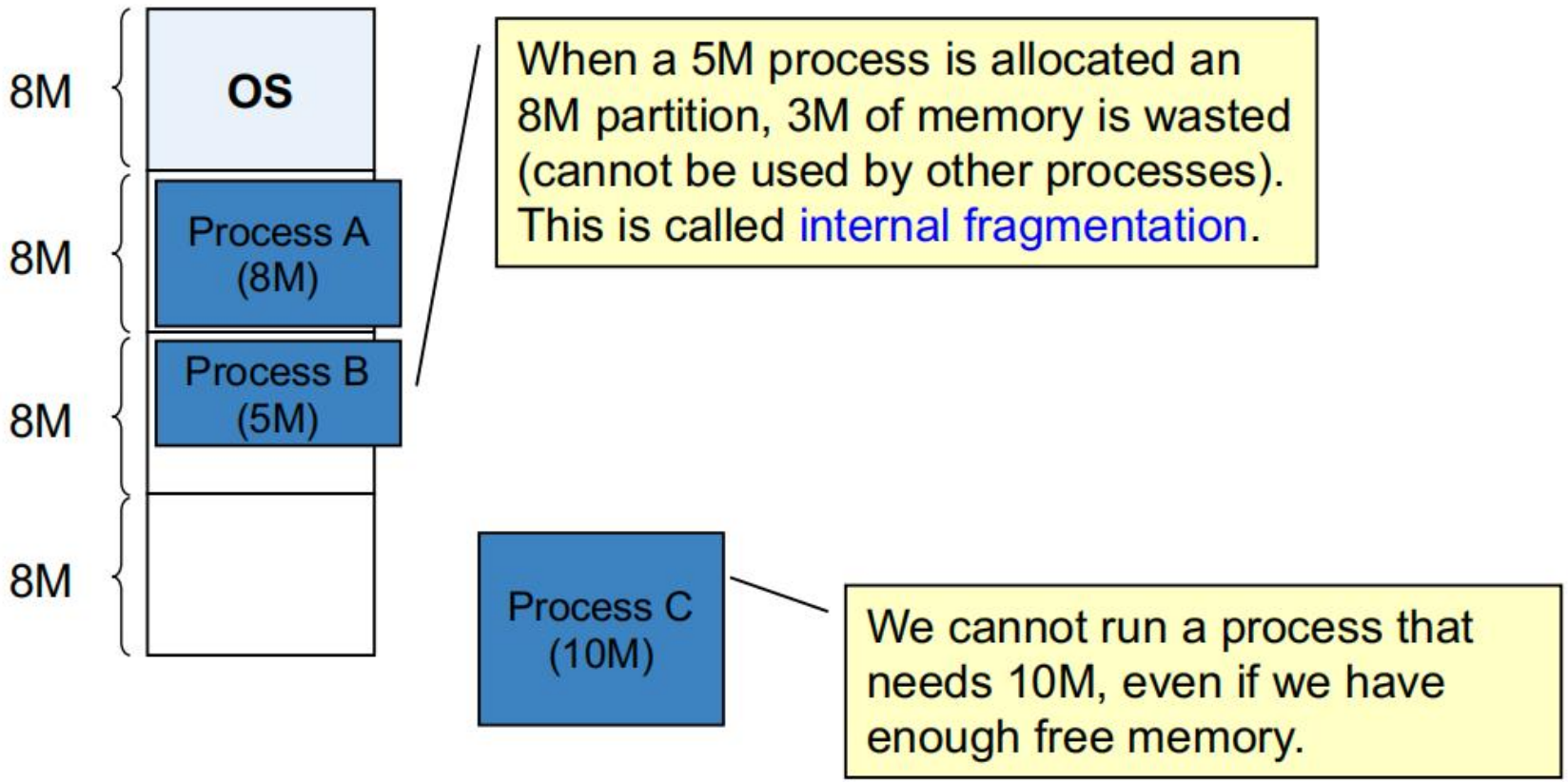
Cons :
- Number of partitions fixed at boot → hard upper-limit for active processes
- Internal fragmentation still exists, poor memory utilization generally

Fixed Partitioning

- Partition available memory into regions with fixed boundary (e.g. at system boot up)
- Each process is allocated ONE partition



Internal Fragmentation

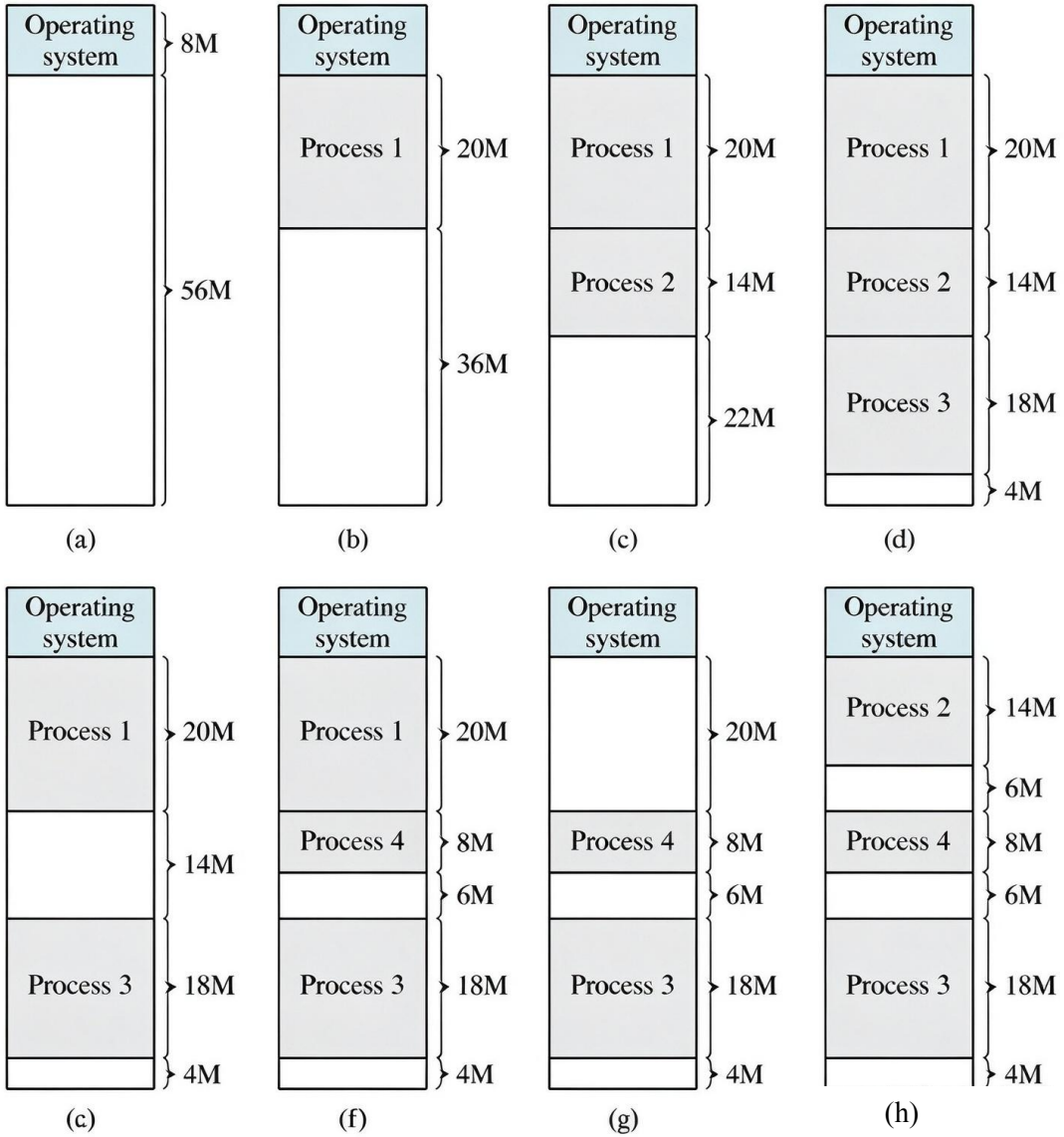


Partition- Dynamic Partitioning



- Partitions are of variable length and number
- Process is allocated exactly as much memory as required
- No internal fragmentation, but there is still external fragmentation
- Must use compaction to shift processes so that all free memory is in one block

Partition- Dynamic Partitioning



We have learned fixed partitioning, its main trouble is internal fragmentation.

To solve this problem, dynamic partitioning is introduced.

Core idea: allocate exactly the memory size that process needs, no extra space.

Historical example: IBM OS/MVT.

Note: dynamic partitioning is also obsolete in modern OS.

Compare with internal fragmentation from fixed partitioning:

- Internal fragmentation: wasted space inside allocated partition (fixed partitioning)
- External fragmentation: wasted space from scattered free blocks (dynamic partitioning)

Partition- Dynamic Partitioning

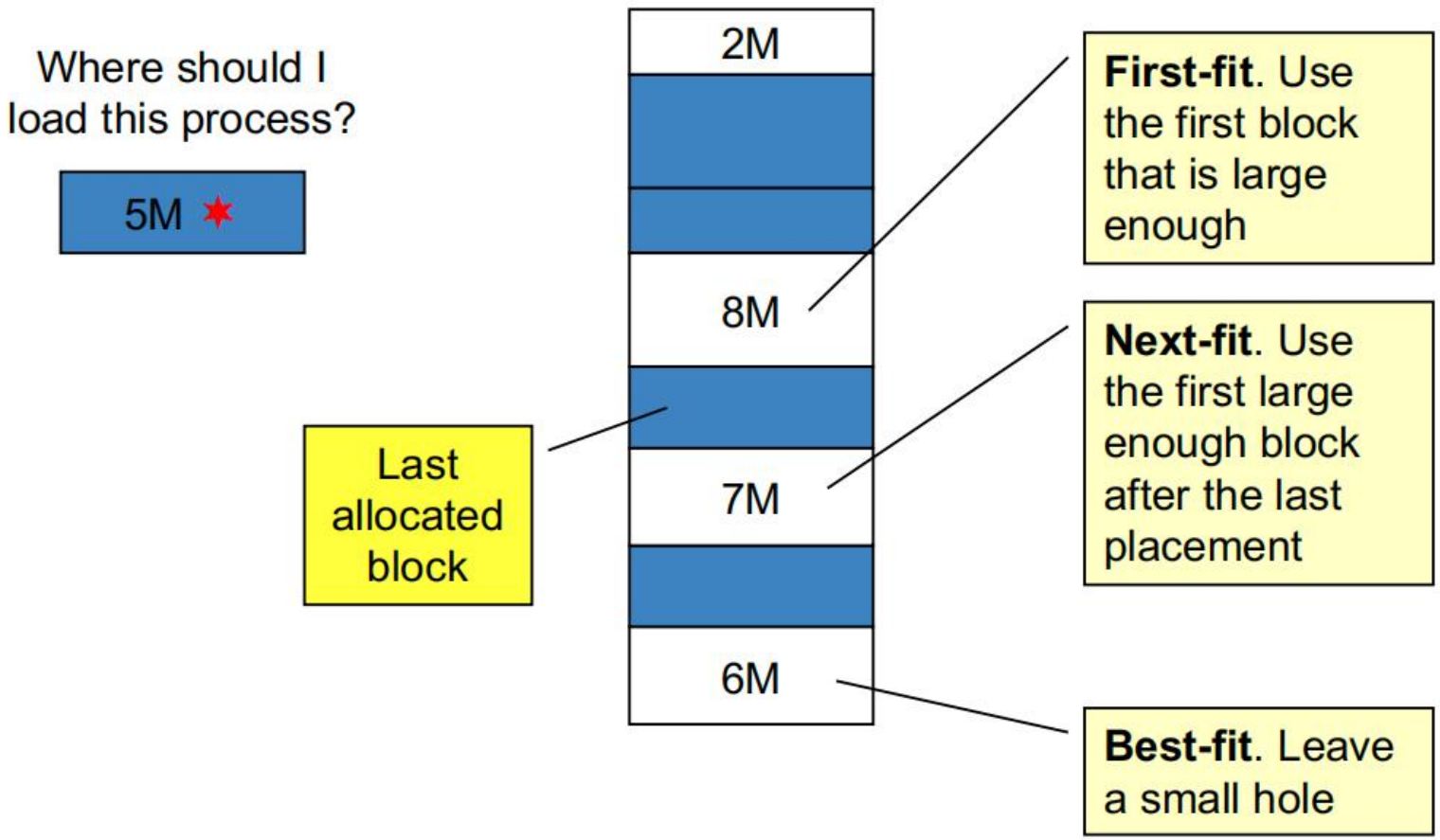
- Fixed partitioning ➜ problem: internal fragmentation
- Dynamic partitioning solves internal fragmentation, but introduces external fragmentation

➢ Remedy 1: compaction (costly, requires dynamic relocation)

➢ Remedy 2: placement algorithms (first-fit / best-fit / next-fit) to mitigate fragmentation

- Finally need replacement algorithm when memory is fully exhausted

Placement Algorithm

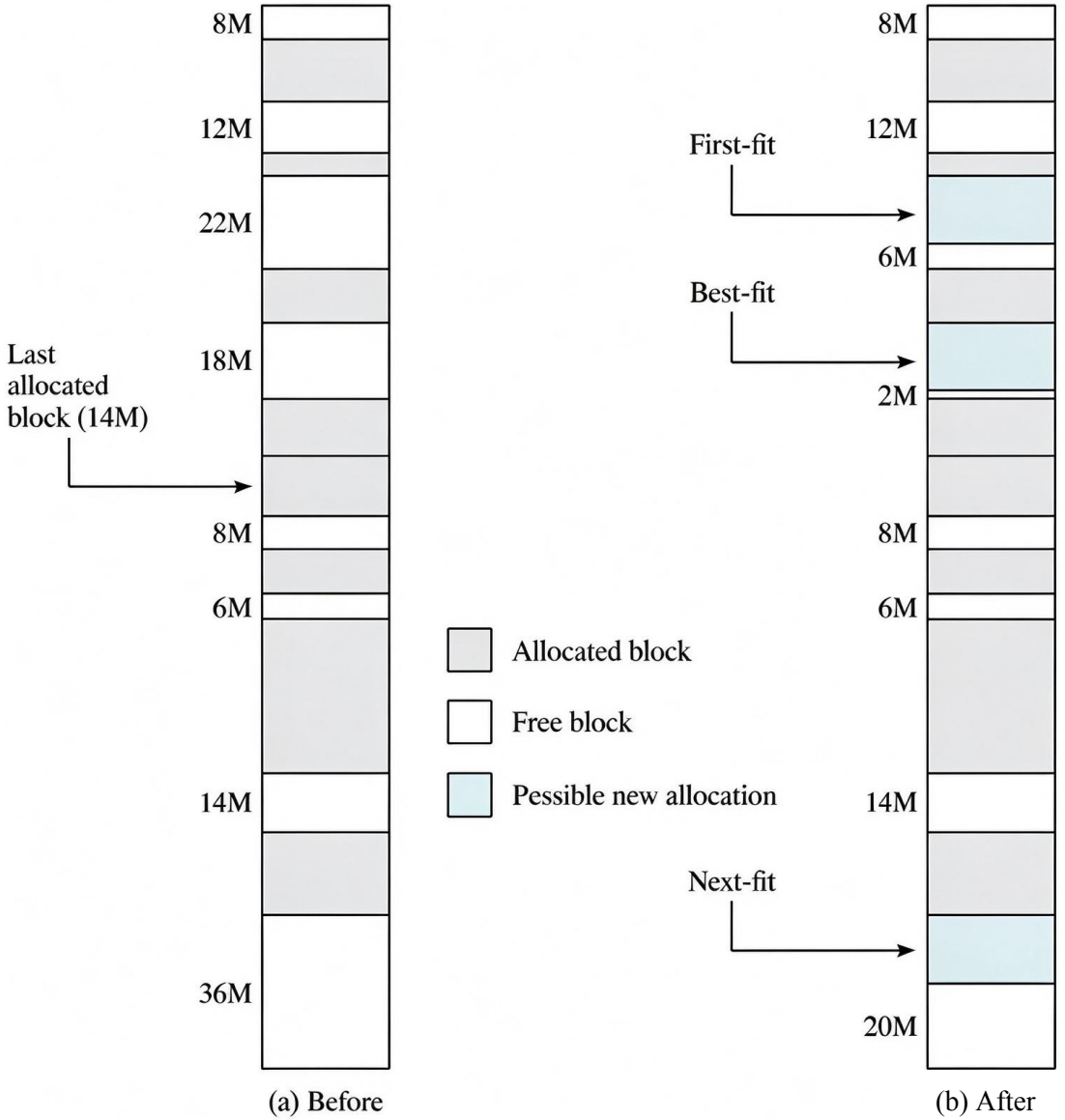


Partition- Dynamic Partitioning

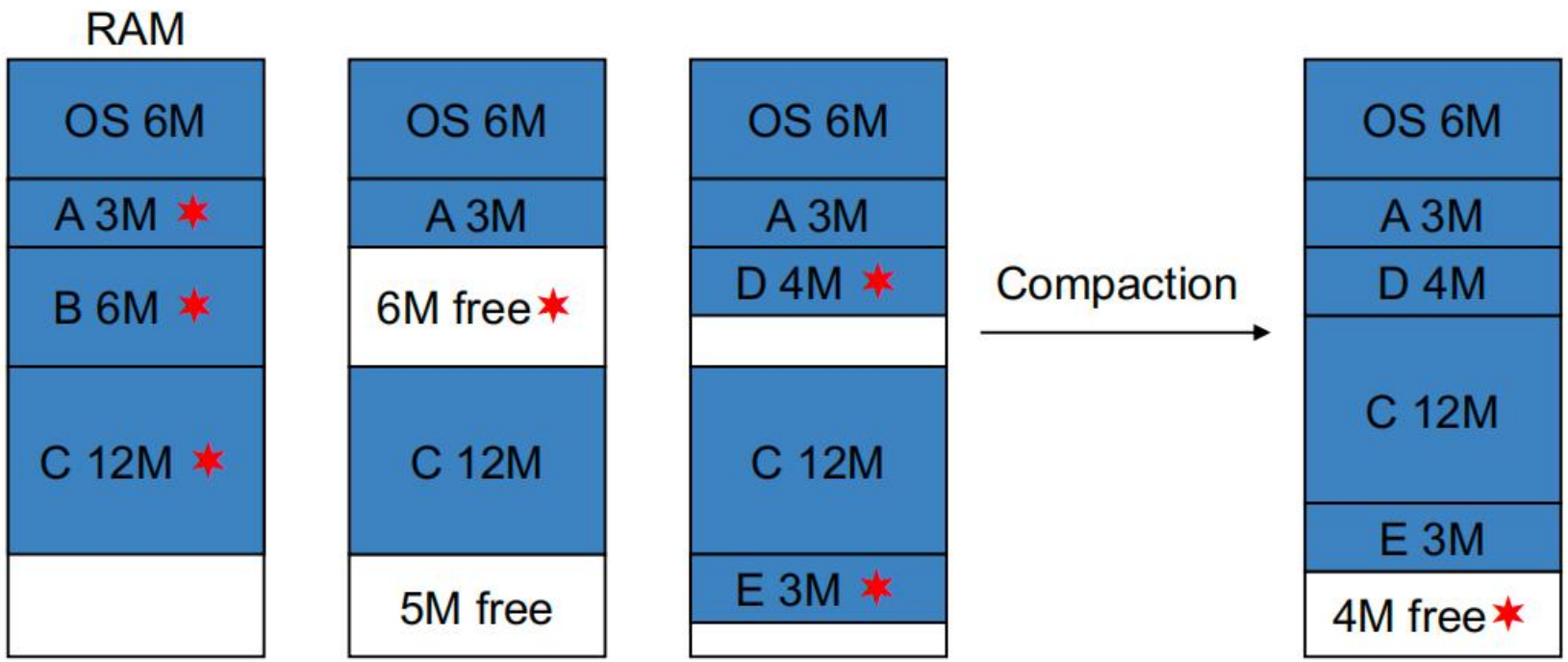
Placement Algorithm

Three algorithms: First-fit, Best-fit, Next-fit.

- First-fit: simplest, generally best & fastest; drawback: many tiny free holes pile up at memory front.
- Next-fit: start scan from last allocation position. Tend to break large tail free block into small pieces → more compaction needed.
- Best-fit: misleading name! NOT the best overall. It leaves many tiny useless fragments, triggers compaction most often.



External Fragmentation



Holes appear as processes are created and terminated. This is called external fragmentation. In this example, we have totally 4M free memory. But there is no contiguous region large enough for a new process. We need to perform compaction.

Paging

Key concepts in Paging

- Physical memory (RAM), Frame, Physical address
- Addressing space of each process, Page, Logical address
- Page table
- Address translation

Paging

Paging is introduced to solve **external fragmentation** problem

Partition Allocation Problems

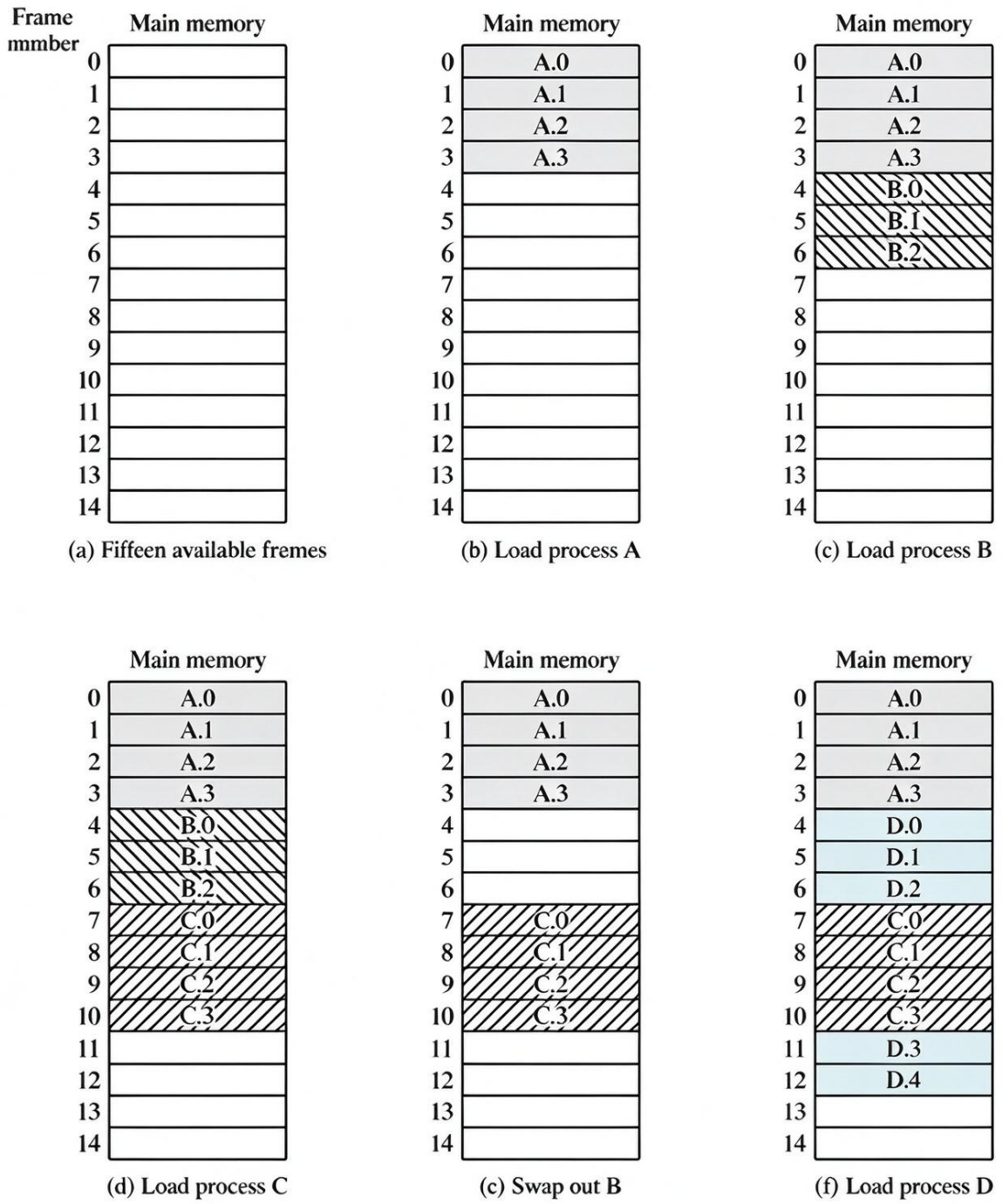
- Fixed-size partition: suffers from internal fragmentation
- Variable-size partition: suffers from external fragmentation

Core Idea of Paging

- Main memory is split into fixed-size frames
- Process is split into fixed-size pages
- Pages can be allocated to non-contiguous frames

Fragmentation property for paging

- Eliminate external fragmentation
- Still has internal fragmentation, only inside the last page



Page Table

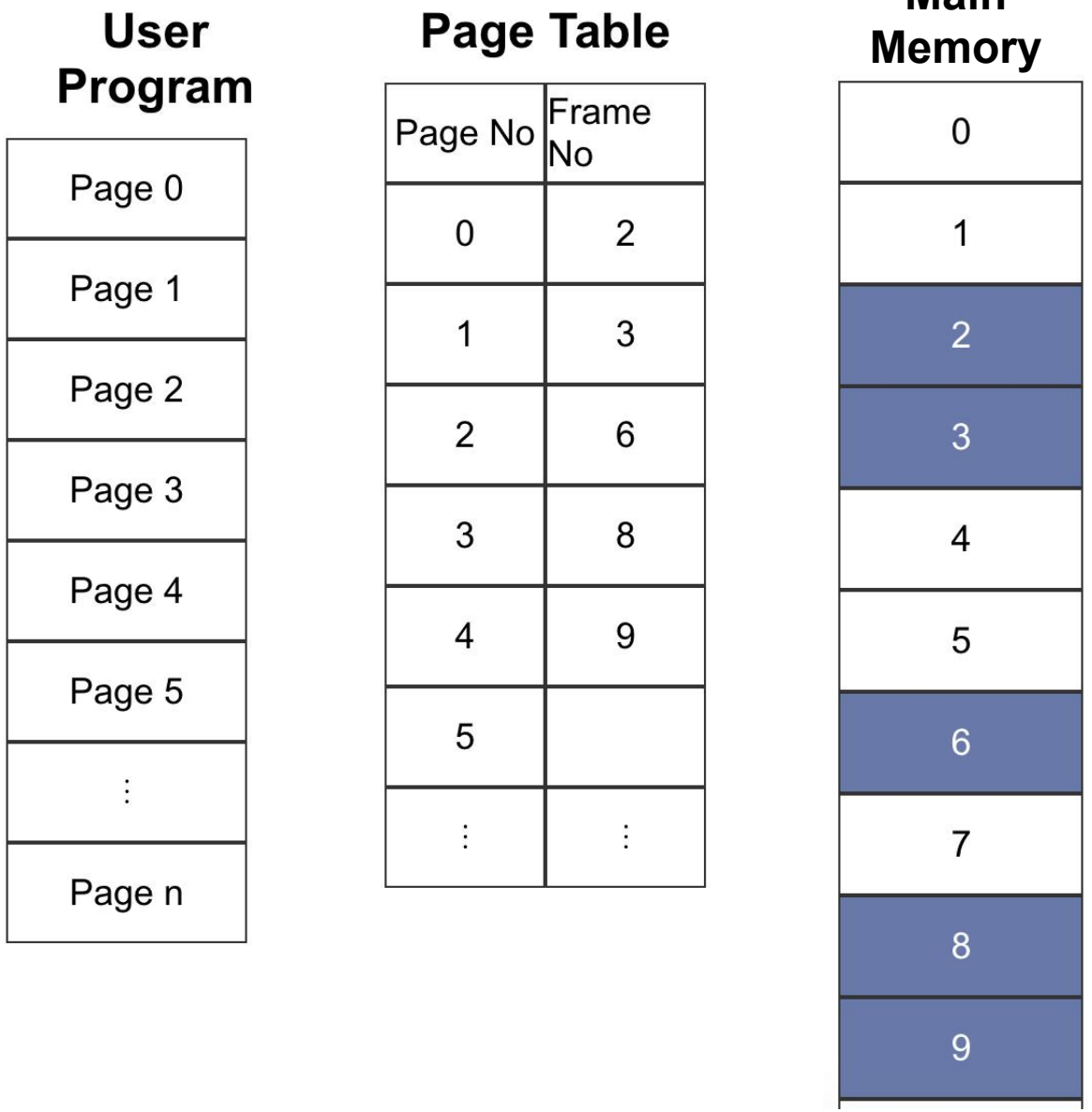
- One page table for each process
- Use page number as index to look-up corresponding frame number

Logical address composition

- Logical address = Page Number + Offset

Address translation is done by hardware

1. Split logical address into page number and offset
2. Look-up page table for frame number
3. Concatenate frame number and offset to get physical address

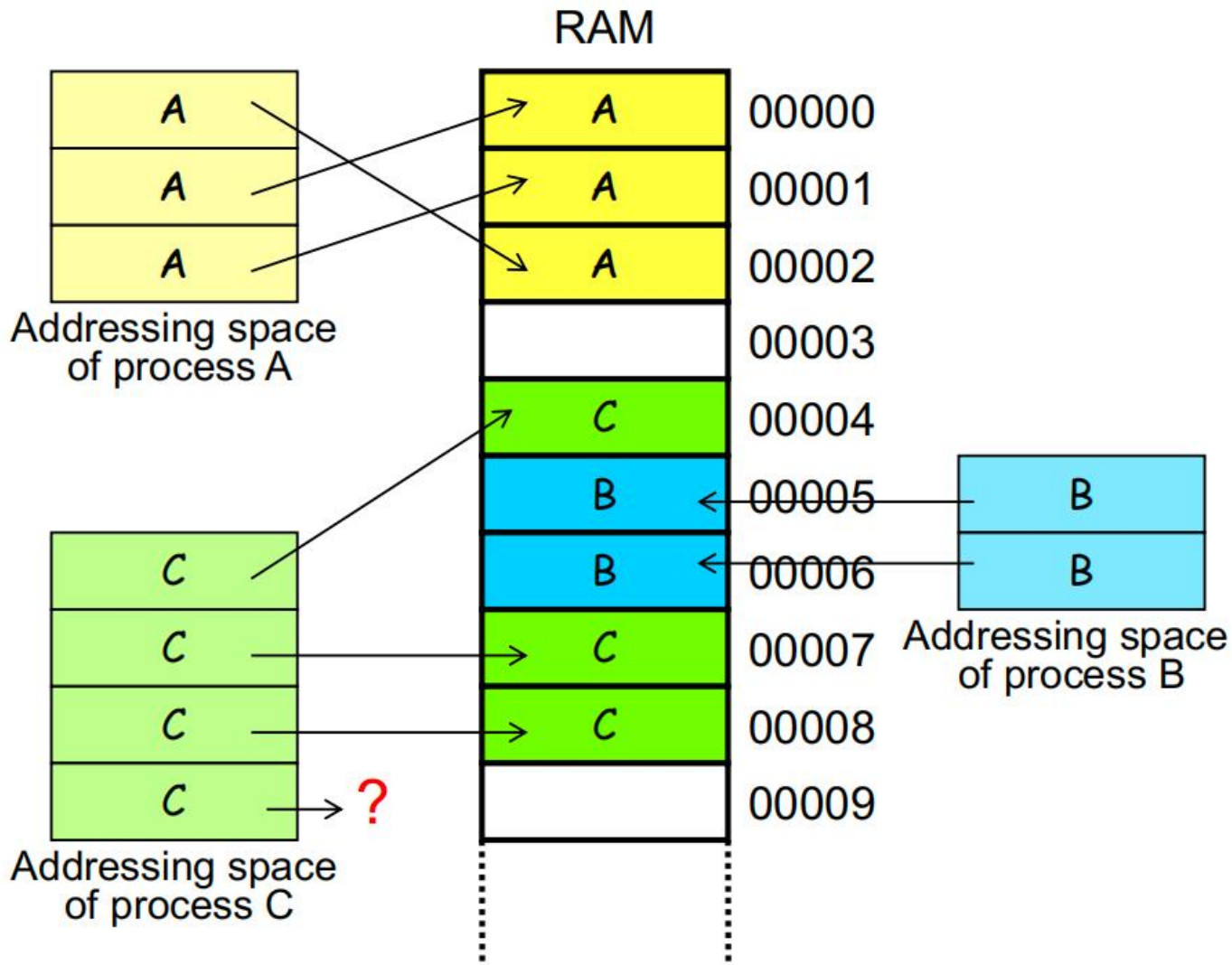


Pages and Frames

The pages in the addressing space of each process are mapped to the frames in the physical memory.

Note that contiguous pages may be mapped to separate frames.

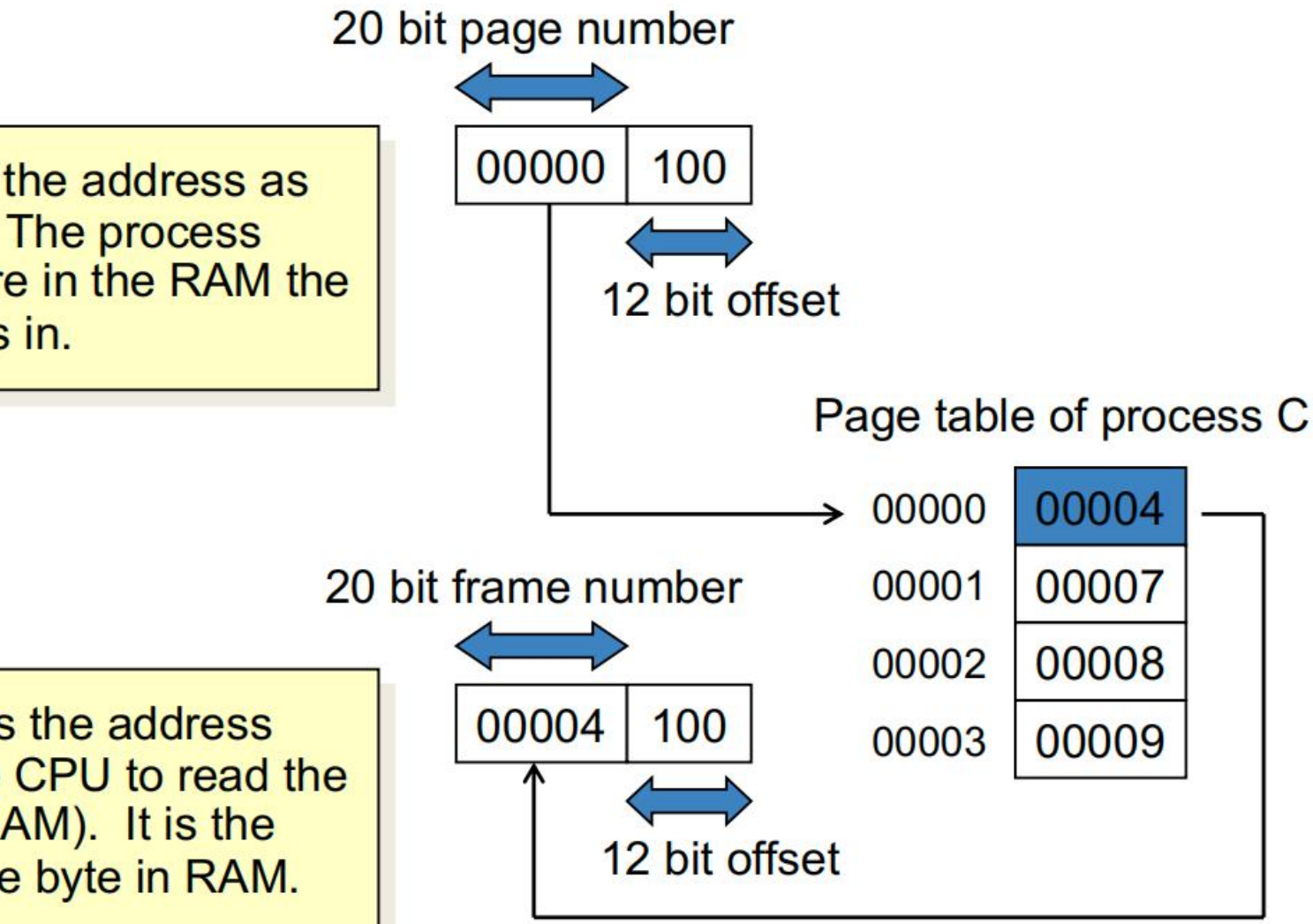
Some pages might not be in RAM at the moment. More about this in chap 8.



Address Translation

Logical address is the address as seen by a process. The process does not know where in the RAM the byte actually resides in.

Physical address is the address actually used by the CPU to read the physical memory (RAM). It is the actual address of the byte in RAM.

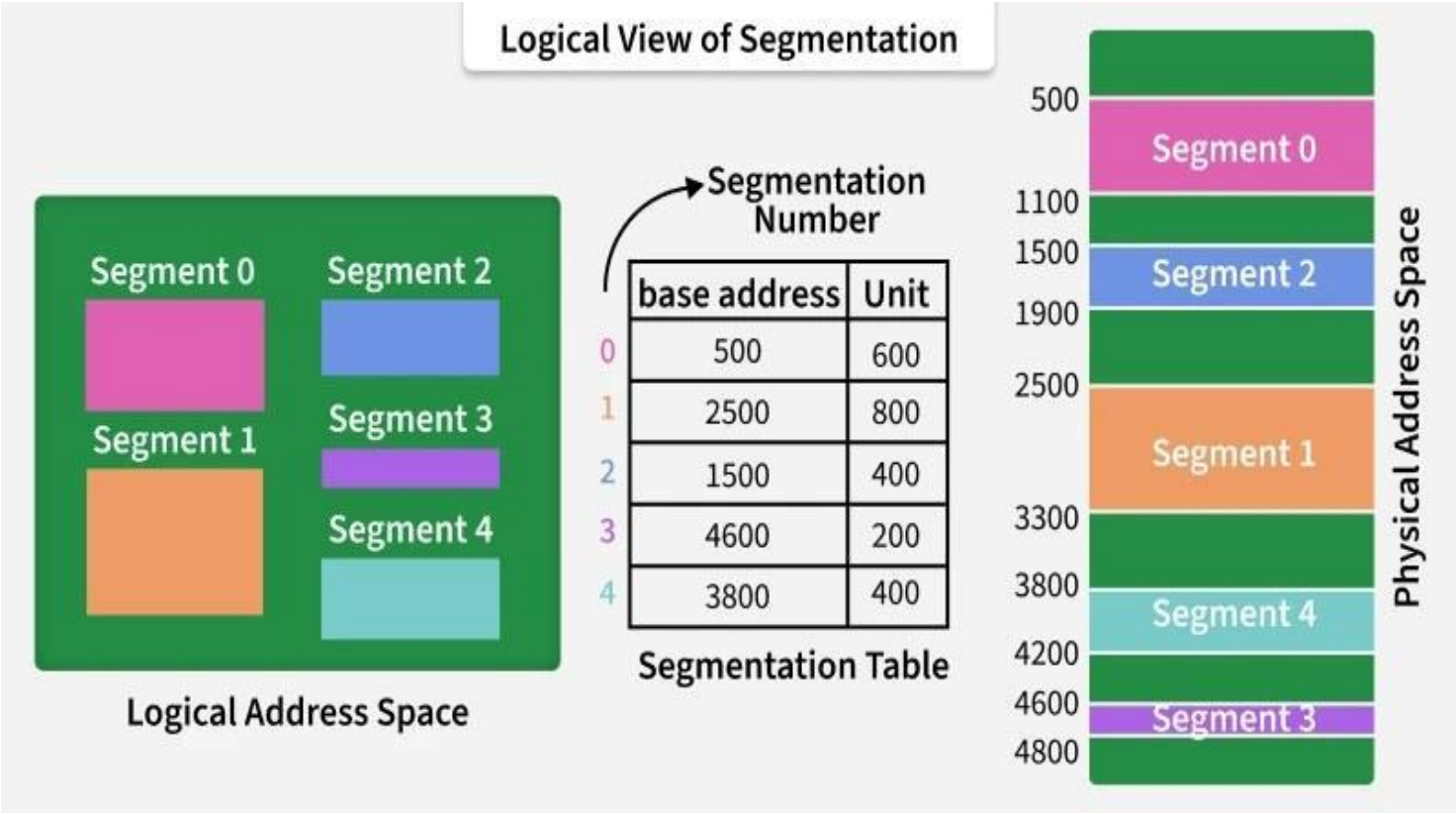


Segmentation

- Program divided into variable-length **segments**
- Logical address = segment number + offset
- Segments can be non-contiguous in memory
- Fragmentation: no internal fragmentation; external fragmentation exists (reduced)
- Visible to programmer / compiler (contrast: paging is transparent)
- Segment Table: per-process, each entry: start address + segment length

Address translation:

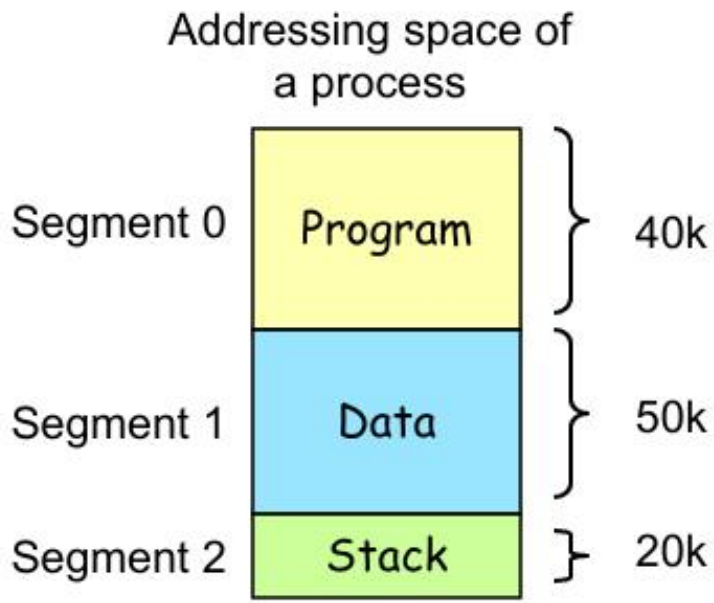
1. Get segment number from logical address
2. Look-up segment table
3. Bounds check for offset
4. Physical address = start address + offset



Segments

The addressing space of a process is divided into parts of unequal length called **segment**s. Each segment usually holds 'data' of a certain type, e.g. program code, global data, and stack. Each segment is identified by a number.

Segments in Pentium is implemented with segment registers (CS, DS, SS, etc) and implicit or explicit association with address register. In concepts, this is similar to what we study here.



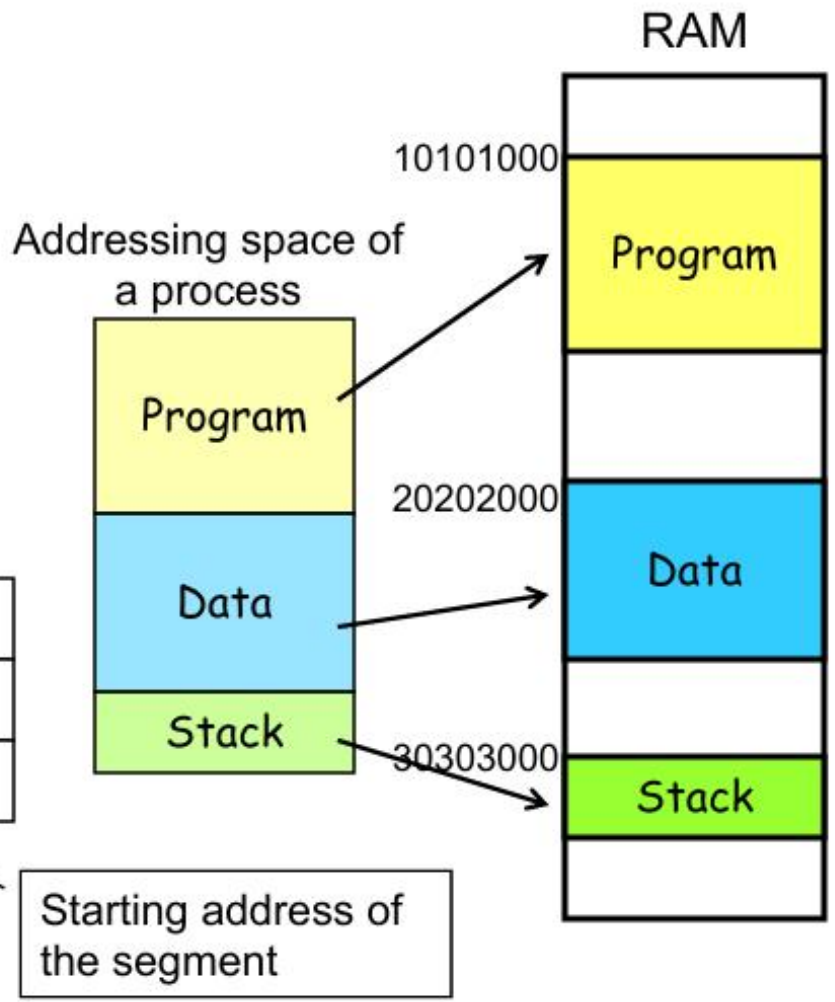
Segment Table

The segments are mapped to the RAM through a **segment table**. It marks the base address and length of the segment.

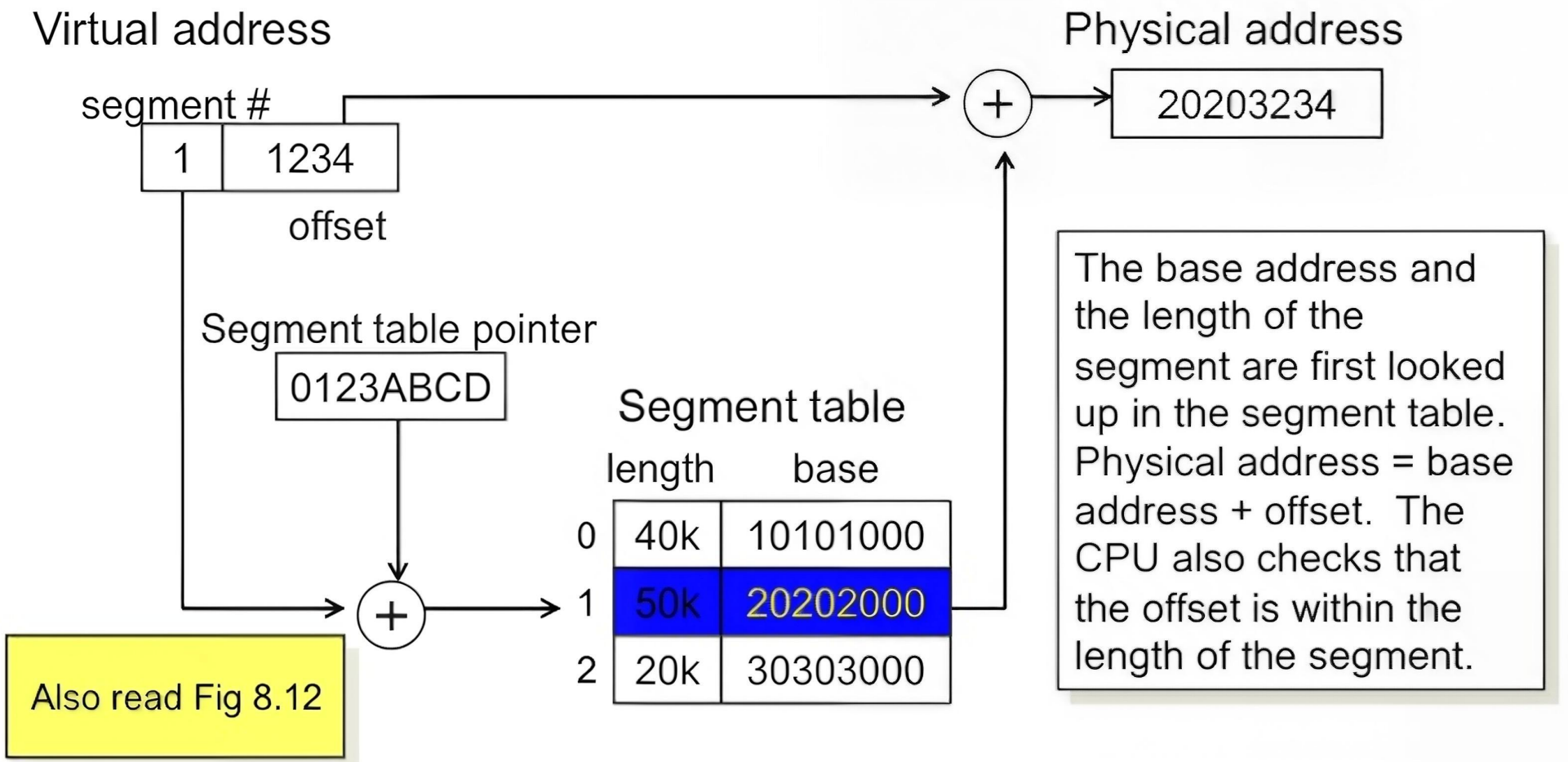
Segment table

	P	M	length	base
0	1	0	40k	10101000
1	1	1	50k	20202000
2	1	1	20k	30303000

Starting address of the segment



Address Translation



Virtual memory

- Separate virtual-address space from physical-address space
- Core idea: only part of process resides in main memory (demand loading)
- Remaining pages are stored on disk as backing store
- MMU still performs address translation from virtual address to physical address
- Page-table has valid-invalid bit:
  - Bit = 1: page is present in main memory, normal memory access
  - Bit = 0: trigger page-fault exception

MMU — Memory-Management Unit
- Hardware component of CPU
- Main function：translate virtual address into physical address

General layout of ***virtual memory***

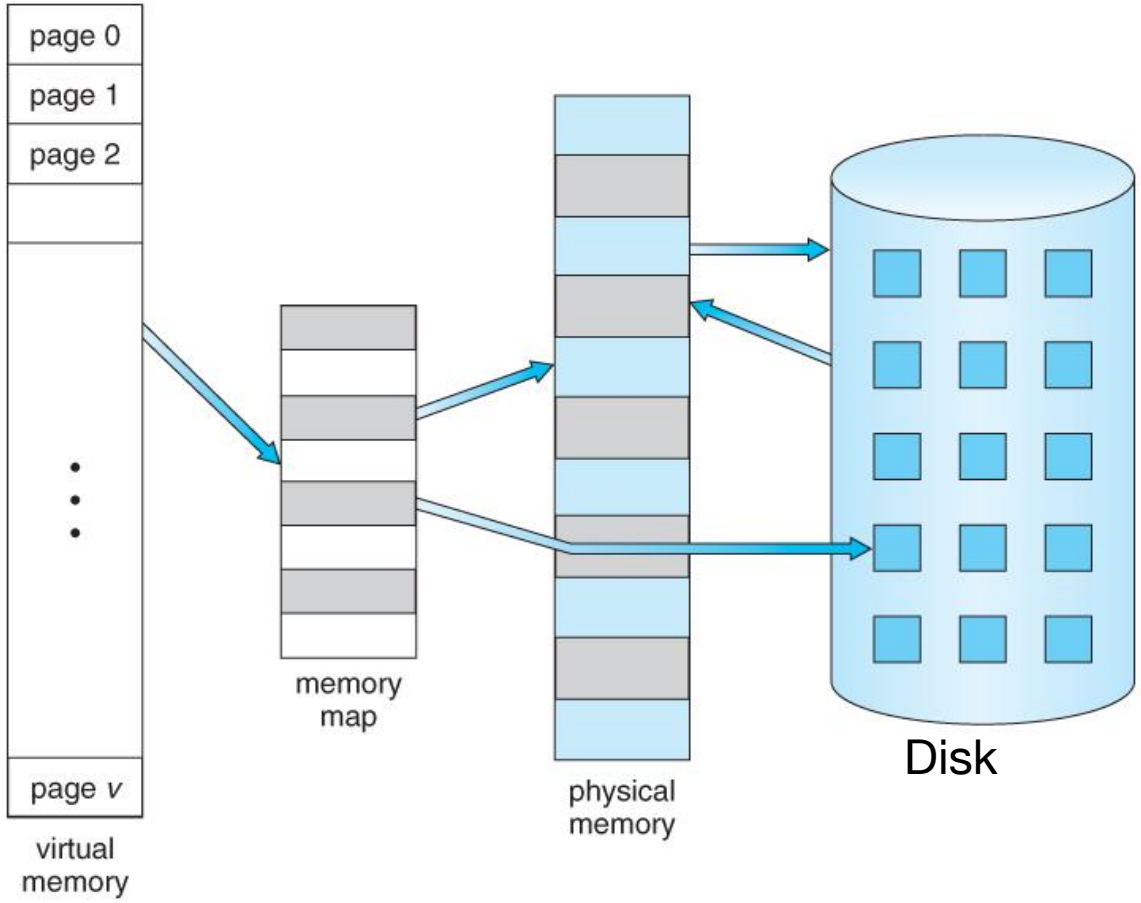
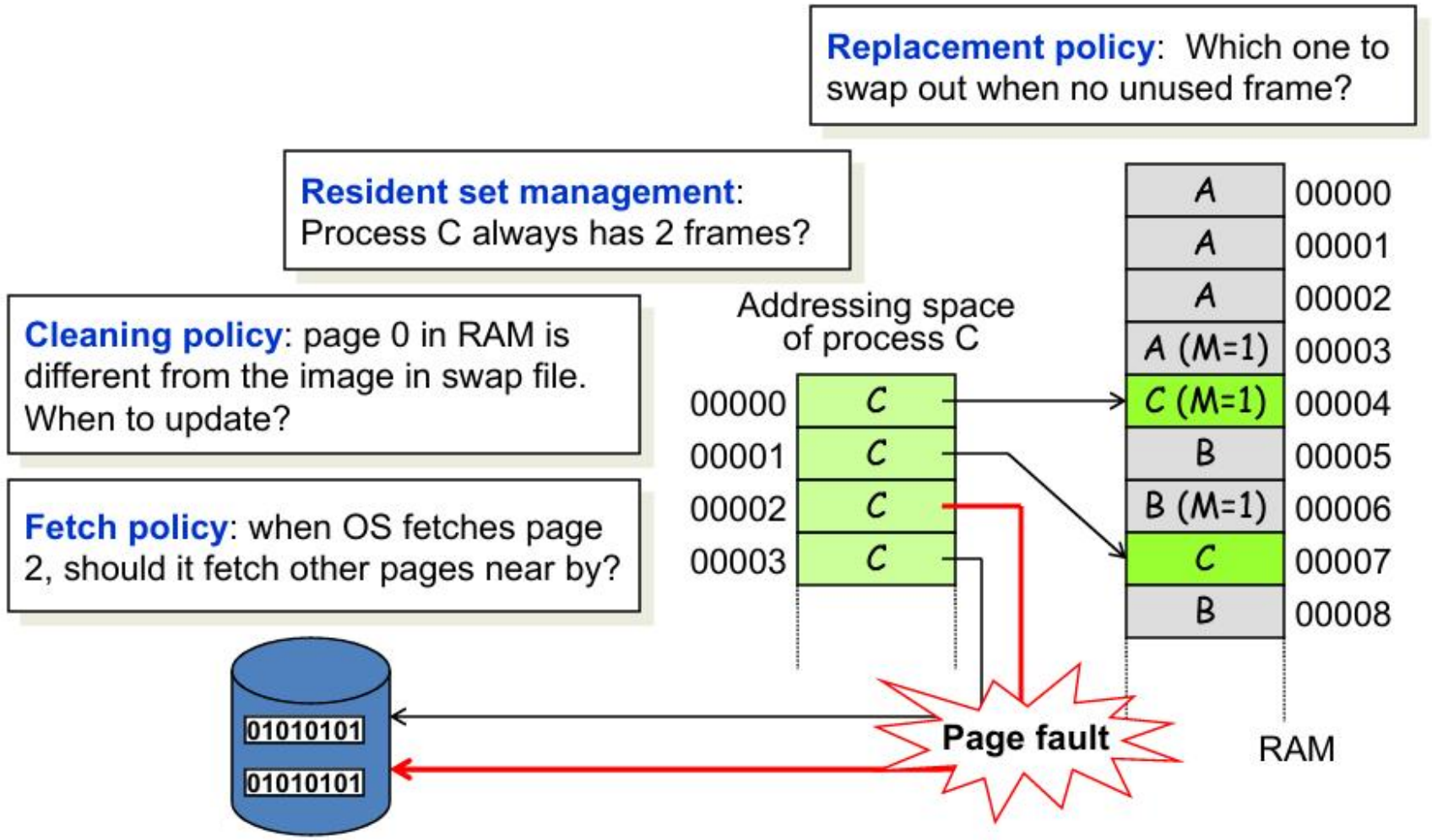
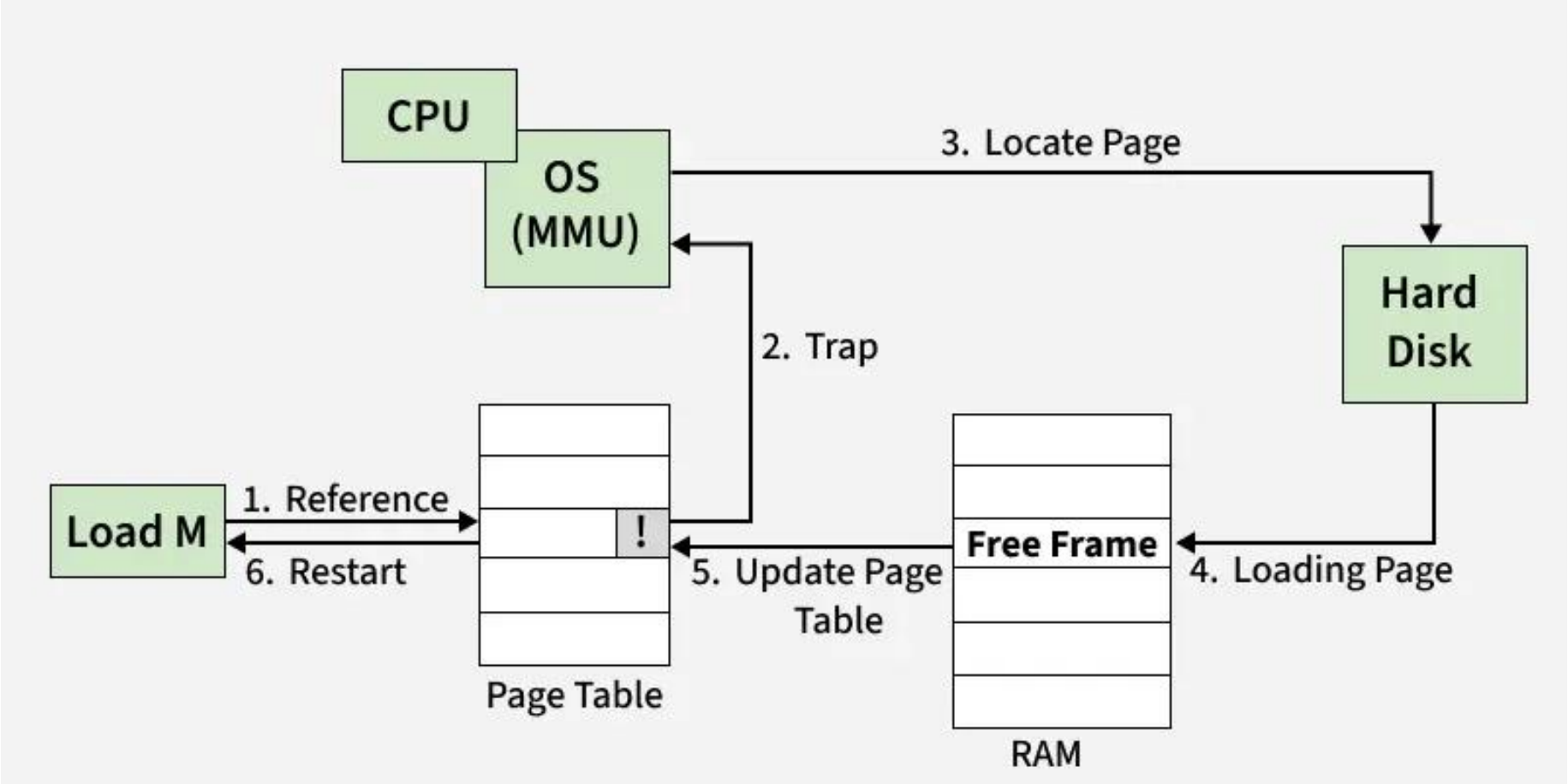


Diagram showing virtual memory that is larger than physical memory

OS Policies for Virtual Memory



Page Fault Handling



Exercise 1

- **Question 1 — Conceptual Understanding**

➢ Briefly explain the key difference between First Fit, Best Fit, and Next Fit in terms of how each algorithm selects a free partition for an incoming memory request.

- **Question 2 — Simulation Problem**

➢ Assume the following free partitions are arranged in address order :

Partition A: 10 KB	Request 1: 12 KB
Partition B: 20 KB	Request 2: 8 KB
Partition C: 5 KB	Request 3: 18 KB
Partition D: 30 KB	
Partition E: 15 KB	

For each algorithm,
(a) First Fit (b) Best Fit (c) Next Fit

show:
1.Which partition is chosen for each request.
2.The state of the free partition list after each request.
3.Which algorithm is most likely to leave many small free blocks that are harder to reuse in the future? Explain why.

Exercise 2

Memory free block list (address start, block size):

Free blocks:

- Start = 100, Size = 120
- Start = 300, Size = 80
- Start = 450, Size = 200
- Start = 700, Size = 150

Sequence of allocation requests (request size):

- Request1: 90； Request2:110； Request3:70

Simulate First-fit, Best-fit, Next-fit allocation.

Assume for Next-fit: initial starting pointer points to the first free block (start=100).

End

- Next

Memory management – hardware
Memory management algorithms – OS support